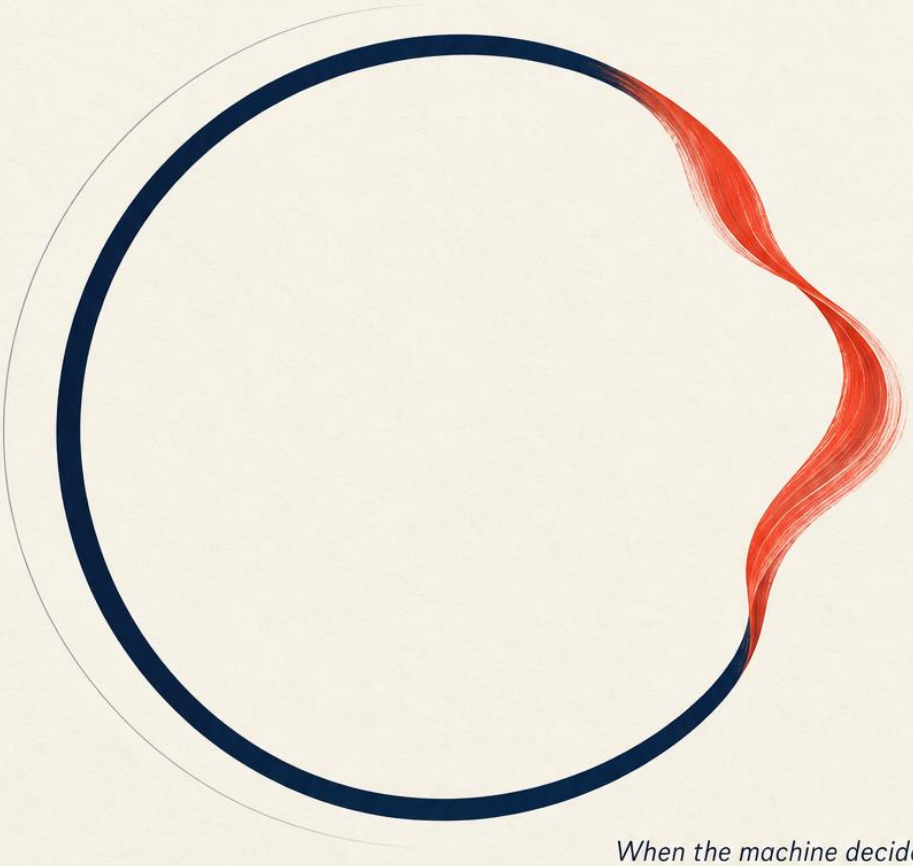
Agentic Loops

Building Software Where Models Decide What Happens Next



When the machine decides,
what keeps it honest?

José Luis Martin Matias

THE GENERATIVIZATION SERIES

BOOK I

THE GENERATIVIZATION SERIES · BOOK I

Agentic Loops

Building Software Where Models Decide What Happens Next

Free sample

This file holds the front matter, the preface, and Chapter 1 and Chapter 2 of Agentic Loops, by José Luis Martin Matias, typeset exactly as they appear in the book, between its two covers. The labs the chapters end with are in the same repository and run without an API key or any third-party package.

https://github.com/jmartinmatias/agentic-loops

THE GENERATIVIZATION SERIES · BOOK I

Agentic Loops

Building Software Where Models Decide What Happens Next

G(body)

José Luis Martin Matias

When the machine decides, what keeps it honest?

Agentic Loops: Building Software Where Models Decide What Happens Next

by José Luis Martin Matias

Book I of The Generativization Series

Every anchor story in this book is documented, and every printed lab output is the output the code actually produced. Meridian, the company that runs through every chapter, is invented. Its problem is not.

This edition was compiled on September 6, 2026 from the chapter sources, the appendices, the labs, and the reference runtime in the Agentic Loops manuscript folder.

For the engineers whose expensive lessons fill these pages,
most of whom never knew they were writing a chapter of somebody's book about AI.

Table of Contents

Preface

I spent two decades in a corner of finance where software is not allowed to be approximately right.

Fund administration is the plumbing under the asset management industry: net asset values calculated to the fourth decimal, transfer agency, depositary oversight, regulatory reporting to authorities who ask precise questions and expect precise answers. It is not glamorous. What it is, is a place where you develop an unusual relationship with the word *guarantee*. In that world, a system that is right ninety-nine percent of the time is not a good system with a small flaw. It is a system that will, on a schedule you cannot predict, produce a number that goes into a document that goes to a regulator, and somebody will have to explain it.

So when models arrived that could decide things, I had the reaction I suspect a lot of people in regulated industries had. Not *this will never work*, which was obviously wrong, and not *this changes everything*, which was true but useless.

Something narrower and more uncomfortable: **if the software is no longer doing what somebody wrote, where did the guarantees go?**

That question is this book.

What I found, and why it took a book

The short version is that the guarantees do not go anywhere. They stop arriving for free.

Classical software gives you an enormous amount you never asked for and never notice. Control flow you can read. A failure you can reproduce. Behavior that cannot exceed what the code says, because the code is all there is. None of that is a feature anyone designed; it is a side effect of the fact that a human wrote every branch in advance. Put a model in the loop and every one of those free gifts is silently withdrawn, and what replaces them, if you do nothing, is a system that is impressive in a demo and unaccountable in production.

The work, then, is to rebuild deliberately what you used to get by accident. That turns out to be a real engineering discipline with real machinery: state that tells the truth about a world that moves, doorways narrow enough that only well-formed actions get through, receipts, gates on the things that cannot be undone, memory

policies, stopping conditions, verification ladders, and an honest meter. None of it is exotic. Almost all of it will feel familiar to anyone who has built distributed systems, which is the most encouraging thing I can tell you: this is not a new kind of engineering, it is engineering you already know, applied at a boundary that is new.

It took a book because the machinery only makes sense as a whole. Any one piece looks like overhead until you have watched the failure it prevents.

How this book is built

Thirteen chapters, each with three things.

A true story at the front. Not decoration. Every principle here was learned expensively by somebody, usually long before anyone thought about AI, and the story is the shortest path to why the principle exists. The alarm that almost ended the first Moon landing. A secretary in 1966 who asked to be left alone with a computer program. Two spacecraft teams and two systems of units. A loom in Lyon. A man who could not form new memories. Forty-five minutes in 2012 that cost four hundred and forty million dollars. An airline that argued in court that its chatbot was a separate legal entity. None of these are invented and none are decorative; each one is the chapter's argument, already made, by reality.

A lab that runs. Every chapter ends in code you execute, and the output printed in the book is the output the code actually produced. The labs need no API key and no third-party libraries. That is a deliberate design choice, not a limitation: the model in these labs is a deterministic stand-in behind the same interface a real model uses, which means the behavior on the page is the behavior on your machine, and the whole book becomes reproducible. When you want the real thing, Appendix A shows you the five lines that swap it in.

Two closing instructions. One practical, usually asking you to break what you just built, because you learn more from the failure than the success. One that asks you to circle something in pencil, because a few of these ideas do not pay off until much later, and I would rather plant them than pretend the book is finished when it is not.

There is a running example throughout: a company called Meridian with a churn problem. It is invented. Its problem is not, and you will build one system for it, from a deterministic loop in Chapter 1 to a metered, verified, crash-survivable harness by Chapter 13.

What this book is not

It is not a framework tutorial. No framework is taught here, deliberately, and by the end you will understand what all of them are doing underneath, which ages considerably better.

It is not a machine learning book. You will not train anything. The model is treated as a component with known failure characteristics, which is the correct posture for someone who has to ship a system around one.

It is not a book about the future of work. There is a real argument to be made about what happens to organizations when this technology matures, and I intend to make it, but not here and not mixed in with the engineering. This book stops where the engineering stops.

A word on the series, and on honesty

This is the first of four books, and I want to be straightforward about that rather than coy.

The argument that runs underneath all four is simple to state: software evolves by taking things that used to be written in advance and letting the running system produce them instead. This book does that to exactly one layer, the contents of a step, and holds everything else still, so you can see clearly what melting one layer costs and what has to be built to pay for it. Book II does it to the actor, Book III to the flow, and Book IV asks what is left.

But Book I has to stand on its own, and I have tried hard to make it do that. If you never read another word of the series, you should finish this book able to build and defend a production agent, which is the promise, and the promise is not contingent on anything I write later.

I will also say plainly what I am not certain about. Nobody, including me, knows how far this goes or how fast. The engineering in these pages I am confident about, because it is mostly old engineering wearing new clothes and because every lab here runs. The larger arc is an argument, offered as an argument, and you should feel free to take the machinery and leave the philosophy.

One practical note. Every term the series relies on, from artifact and emission to the frozen core, is defined in this book at first use, and all of them are collected in one glossary, Appendix E of Book IV.

Who I hope reads it

Engineers who have built an agent that worked beautifully in a demo and then did something inexplicable on a Tuesday. Architects who have to sign their name under a system whose behavior nobody wrote. Anyone in a regulated or consequential industry who is being asked whether this technology can be trusted, and who wants a better answer than a feeling.

That last group is where I started, and it is who I was writing for when the writing got difficult.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, emphasis, filenames, and the titles of works.

`Constant width`
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, data types, environment variables, statements, and keywords.

Bold
: Marks a term at the moment it is defined, and the occasional sentence the argument turns on.

Shaded boxes hold sidebars: a distinction, caveat, or piece of history that stands beside the argument rather than inside it. Each chapter opens with an epigraph and a true story, ends with a lab, and closes with two instructions, one practical and one in pencil, followed by a short summary under the heading “Where We’ve Landed.”

Using Code Examples

Every chapter ends with a lab, and every lab runs. The thirteen labs and the reference runtime, `meridian_runtime.py`, accompany the manuscript as runnable files, and they are reproduced in full in this book: the runtime in Appendix A, the labs in Appendix I. None of them needs an API key or a third-party package; Python 3.10 or newer is all that is required. The only optional dependency is the `anthropic` package, used solely by the runtime's `--live` flag.

```
./run_all_labs.sh                              # every lab plus the runtime
python3 labs/ch09_the_exoskeleton_lab.py       # any chapter's lab
python3 meridian_runtime.py                    # the reference runtime
python3 meridian_runtime.py --crash-after 2    # kill mid-run, then resume
python3 meridian_runtime.py --live             # needs ANTHROPIC_API_KEY
```

The experiment to run first is `--crash-after 2`. Watch the world counters before and after the crash. They do not change. That is the whole book in two lines of output.

Thanks

To the engineers whose expensive lessons I have borrowed throughout, most of whom never knew they were writing a chapter of somebody's book about AI. And to the reader who checks the labs actually run. You are the reader I wrote them for.

Now: Chapter 1, and a computer alarm, twelve hundred meters above the Moon.

PART I

The Machine That Decides

CHAPTER 1

One Small Step

Why the loop is the program

> *“Program alarm.”*
>
> *“It’s a 1202.”*
>
> Neil Armstrong and Buzz Aldrin, descending, July 20, 1969

Five minutes into the final descent, with the Moon filling the windows and the engine burning, the computer flying Apollo 11’s lunar module said, in effect: *I have too much to do.*

It said it as a number. **1202** flashed onto the display, a program alarm, a code no astronaut had a page for, and the master alarm sounded in the cabin. Neil Armstrong, a man whose heart rate the flight surgeons watched climb only twice in the whole mission, asked Houston for a reading on it. He and Buzz Aldrin were dropping toward the Sea of Tranquility in a machine with less memory than a short email, and the machine had just interrupted its own landing to complain.

You will not appreciate what happened next until you know what was happening inside the box.

The Apollo Guidance Computer was, by any modern measure, almost nothing: around seventy pounds of hand-wired electronics, a couple of kilobytes of erasable memory, a clock several thousand times slower than the phone in your pocket. Its software had been written over years at the MIT Instrumentation Laboratory by a team whose software engineering division was led by Margaret Hamilton. Written, and then literally *woven*, thread by thread through magnetic cores, by textile workers whose finished fabric was the program. If you wanted to change a line of code, you re-wove the rope.

And at its heart, that software was a loop. Not a metaphorical loop: an executive scheduler, built on a design by Hal Laning, that cycled through a queue of jobs in strict priority order. Read the radar. Update the trajectory. Fire the thrusters. Refresh

the display. Around and around, many times a second, each job hand-written, each transition known in advance, the whole choreography fixed months before launch and frozen (in the ropes, literally) for the flight.

The 1202 alarm meant the loop was drowning. A checklist decision had left the rendezvous radar, the one you would need to find the command module again in an emergency, switched on during descent, and a subtle electrical mismatch made it flood the computer with meaningless interrupts, stealing roughly fifteen percent of its processing time. Fifteen percent does not sound fatal. But the descent schedule had been budgeted close to the bone, and the executive was now being asked to do more work per cycle than the cycle contained. Its job queue overflowed. Hence the alarm: 1202, executive overflow.

Here is the part that matters. The computer did not crash. It did not freeze, did not corrupt the trajectory, did not do the thing every machine you have ever owned does when it runs out of headroom. It executed a plan its authors had written for exactly this moment, years earlier, on the ground: it **restarted itself, flushed everything from the queue, and rebuilt only the jobs that mattered**, highest priority first. Steering the descent: kept. Navigation: kept. The display refresh and the low-priority housekeeping: dropped without ceremony. The computer triaged itself, mid-landing, in a fraction of a second, and kept flying.

In Mission Control, the question "GO or abort?" landed on a guidance officer named Steve Bales, age twenty-six, who turned to a backroom engineer named Jack Garman, age twenty-four. Weeks earlier, in a simulation, this exact family of alarms had been thrown at the team and they had called an abort. Wrongly, the debrief concluded. So Garman had done a very human thing: he had written out, by hand, a cheat sheet of every program alarm and what it actually meant. He looked at his list. As long as the alarms did not come continuously, the computer was doing precisely what it was designed to do. Bales called GO. Capcom Charlie Duke passed it up ("we're GO on that alarm"), and the alarms kept coming, four more of them, all the way down through the pitchover, while Armstrong took over the final approach by hand to skip past a boulder field, and Eagle landed with well under a minute of hover fuel to spare.

Half a billion people heard "one small step." Almost none of them heard the other sentence, the one this book is built on, the reason a twenty-six-year-old could bet two lives and a decade of national effort on a machine that was actively complaining:

He knew what the loop would do.

Every job the computer could run had been written by a person. Every transition between jobs had been decided in advance. And when the authored behavior hit conditions nobody had scripted, a *separate layer* (priorities, restart tables, a watchdog) guaranteed the outcome anyway. The behavior was frozen. The guarantees were locked. That is what made GO a rational word.

This book is about what happens when we thaw the first of those layers: when the next step of a program is *generated* by a model at runtime instead of written by a person in advance. It is a genuinely new way to build software, and it is the reason you picked this book up. But you cannot understand what changes until you can name, precisely, what used to be fixed. So this first chapter is about the machine we are leaving: the classical loop, in full. We will take it apart, name its four layers, meet it in the wild, and build one: a small, honest, deterministic loop that we will spend the rest of the book melting, one layer at a time.

By the end of the chapter you will own four words (**body**, **actor**, **flow**, **guarantee**) and they will carry you through four books.

1.1 The Oldest Trick in Computing

Strip away sixty years of frameworks and there are only a handful of shapes a program can take. In 1966 (the same year, pleasingly, that a chatbot first fooled a secretary, but that is Chapter 2) two Italian computer scientists, Corrado Bohm and Giuseppe Jacopini, proved a result now folded so deeply into practice that nobody cites it anymore: *any* computation can be expressed with just three structures. Do this, then that (**sequence**). Do this *or* that (**selection**). Do this *again* (**iteration**).

Sequence and selection are clerks. They execute and they choose, and then they are finished. Iteration is different in kind. Iteration is the only structure that says *keep going until something is true*, the only one that lets a fixed, finite text produce unbounded, open-ended work. A thousand lines of woven rope flew a spacecraft for days, not because the rope was long, but because it looped.

That is the trick, and it is genuinely the oldest one. The loop is where a program stops being a recipe and starts being a *process*: a thing that persists, watches, responds, and, critically, must one day decide it is done. Recursion, the loop's elegant cousin, plays the same trick wearing academic dress: a function that continues by invoking itself, with a base case standing where the loop's exit condition would stand. Same trick, same obligations.

Two obligations, to be exact, and they will follow us through every chapter of every book in this series:

1. **Something must change each time around.** A loop that revisits the same state has stopped computing and started spinning. (Hold that thought until section 1.7, where we will watch it happen.)
2. **Something must eventually be true.** A loop with no reachable exit condition is not a process; it is a hostage situation. The sorcerer's apprentice enchants the broom in Chapter 10, but the broom is already standing in the corner of this one.

Everything else (the schedulers, the event systems, the orchestration frameworks, the agent runtimes that this series is really about) is engineering wrapped around those two obligations.

1.2 Programs as State Transitions

Here is the least glamorous and most load-bearing idea in this book. A running program, at any instant, can be described by its **state**: everything it knows and everything it has done that still matters. And a loop is nothing more than a rule applied to that state, over and over:

```
while not finished(state):
    state = f(state)
```

That is it. That is the physics. `f`, the **transition function**, takes the world as the program understands it and returns the world one step later. Everything a loop will ever do is contained in three decisions someone made in advance: what counts as state, what `f` does, and what `finished` means.

Look at that tiny program long enough and it separates into layers, the way a landscape separates into geology. Four questions, four layers:

What happens during a step? That is the **body**: the contents of `f`. On Eagle, the body was guidance equations and thruster commands, derived by engineers and checked by other engineers. In your codebase, it is the functions the loop calls.

Who, or what, performs the step? That is the **actor**: the thing with an identity, capabilities, and permissions that executes the body. The AGC and its scheduled jobs. Your worker process. The service account it runs as.

How are steps arranged over time? That is the **flow**: the order, the branching, the scheduling, the choreography. Laning's priority executive. Your pipeline definition. The arrow in every architecture diagram you have ever drawn.

What must remain true, no matter what? That is the **guarantee**: the layer that does not *do* anything. It *forbids* and *verifies*. The priority ordering that said steering outranks displays. The restart tables. Garman's handwritten sheet, which was a guarantee wearing a human face.

Draw them as a stack and label the Apollo column, because we will be redrawing this picture for four books:

```
 layer      | on Eagle, July 1969             | authored when?
------------+---------------------------------+----------------
 GUARANTEE  | priorities, restart, watchdogs, |  design time
            | Garman's list, Bales's GO rules |   (LOCKED)
 FLOW       | the executive's schedule        |  design time
 ACTOR      | the AGC and its jobs            |  design time
 BODY       | guidance equations, commands    |  design time
```

Read the right-hand column and notice what is remarkable about classical software, precisely because nobody ever remarks on it: **every layer is authored before execution.**

The body, the actor, the flow: all of them are *artifacts*, fixed in advance, as thoroughly frozen as software woven into rope. At runtime, nothing new comes into existence; the machine merely *replays* decisions people already made. This is why Bales could say GO. It is why your on-call engineer can read a stack trace. Determinism is not a feature of classical software; it is the medium the whole thing is sculpted in.

Keep the word **frozen** close. In section 1.4 we will make it precise, and in Chapter 2 we will apply heat.

Four words for four books

Body: what happens during a computational step.
Actor: who or what performs the step.
Flow: how steps and actors are arranged over time.
Guarantee: what must remain true, or be verified, regardless of the other three.

This series tells one story: the first three layers, one by one, stop being written in advance and start being generated at runtime, while the fourth absorbs the responsibility they leave behind. This book melts the body. Its sequels melt the actor, then the flow. The guarantee is the layer we will fight, for four books, to keep frozen.

1.3 Everything Is Secretly a Loop

Once you have the shape, you see it everywhere. And I do mean everywhere, because the software industry has spent sixty years rediscovering the loop and each time giving it a more dignified name.

Cron is a loop with a wristwatch: wake, check the schedule, run what is due, sleep.

An ETL pipeline is a loop wearing a lanyard: extract, transform, load, and back tomorrow at 02:00 to do it again. **A web server** is a loop that answers the door: accept a connection, handle the request, return to the doorway. The fact that it may be doing so on ten thousand doors at once changes the engineering enormously and the shape not at all. **A game engine** runs the most honest loop in the business: read input, update the world, draw the world, sixty times a second, forever, with a frame budget so strict it makes Apollo's executive look leisurely. **The event loop** in your browser and in Node.js is a loop that has outsourced its to-do list: it does not know what work is coming, only how to take the next callback from the queue and run it. **A thermostat**, and every control system descended from it up to and including the one that flew Eagle, is a loop with a grievance: measure the world, compare it to the world you wanted, act to shrink the difference, measure again.

And **workflow engines** (the Airflows, Step Functions, and Temporal clusters of the world, the tools closest to where this series is heading) are loops that have been promoted into management. You hand them a graph of tasks, and underneath the DAGs and the retry policies there is a scheduler doing exactly what Laning's executive did: cycle through pending work, pick the next eligible job, dispatch it, record what happened, go around again.

Even the machine beneath all of this is a loop. Fetch the next instruction, decode it, execute it, advance the counter: every processor ever shipped runs one loop, forever, and everything you have ever called "software" is just cargo riding inside it.

Here is why this tour matters and is not just a party trick. In every single example, you can point cleanly at the four layers. Cron's *flow* is the crontab; its *bodies* are your scripts; its *guarantee* is, notoriously, little more than an exit code and hope. The game engine's *guarantee* is the frame budget, enforced with a ruthlessness Apollo would recognize. The workflow engine's entire commercial value is that it took *flow* and *guarantee* (ordering, retries, timeouts, exactly-once-ish execution) out of your application code and made them someone else's rigorously tested problem. Different decades, different logos, same anatomy. The industry has been shipping the same four-layer machine since 1969 and arguing only about which layer to sell.

Which sets up the real question, the one the rest of this book exists to answer: if the anatomy never changes, what exactly is new now? Not the loop. The loop is fine. What is new is *who writes the inside of it*.

If you arrived with a different vocabulary

The tooling world has its own names for the things this series names, and it is worth mapping them once, because the four layers are underneath all of them. An **agent harness**, the fixed machinery that gives a model its tools, its state, its permissions, and its loop, is this book's subject from Chapter 9 onward, where it is called the exoskeleton: actor and flow held frozen around a generated body. **Evals** are the guarantee layer pointed at a component whose behavior is generated rather than written, and Chapter 12 is about what that costs when the thing under test is a distribution rather than a function. A **gauntlet loop**, run the agent until its output survives a battery of checks, is a stopping condition and a verifier bolted together; Chapter 10 is about the first half, Chapter 12 the second, and the question the pattern rarely asks, who wrote the checks and whether the thing being checked can see them, is the subject of Book III. And a **self-improving agent**, in the strong sense of a system that rewrites its own harness, is the one thing the series argues should not exist, on structural grounds that take until Book III, Chapter 13 to earn. The words change. The layers do not.

1.4 What "Frozen" Means

Time to make our central metaphor precise, because we are going to lean on it for four books.

Every piece of a software system belongs to one of three categories, and the sorting question is simply: *when does it come into existence?*

An **artifact** exists *before* execution. Someone wrote it, reviewed it, versioned it, deployed it. Your source code. The crontab. The DAG definition. The woven rope. Artifacts are **frozen**: at runtime they can be read and replayed but not created. Their defining virtue is that you can study them in advance (test them, prove things about them, print them out and defend them to a review board) because they hold still.

An **emission** exists *only during* execution. It is produced by the running system, in response to conditions no author fully foresaw: log lines, computed values, a game's particular frame, this particular HTTP response. Emissions are **fluid**. Classical software engineering's quiet genius was to arrange things so that emissions were *boring*: mere consequences, fully determined by frozen artifacts plus inputs. If an emission ever surprised you, that was called a bug.

A **guarantee** is the third thing, and it is neither. It is a *constraint or verifier applied to emissions*: the type check, the schema, the assertion, the priority table, the watchdog. Guarantees are **locked**: frozen artifacts, yes, but with a special role. They do not produce behavior; they *bound* it. When the rendezvous radar produced conditions no one had scripted, it was not the body that saved the landing. Bodies can only do what they were written to do. It was the locked layer.

Now the fulcrum of the whole series, stated plainly. In classical software, *body, actor, and flow are all artifacts.* All frozen. And this book, the one in your hands, is about applying a single transformation to the first of them:

```
G(x):  artifact(x)  ->  emission(x)
```

G for *generativize*: take something that used to be authored in advance and let the running system produce it. This book performs `G(body)`: the contents of each step, the decision about *what happens next*, becomes an emission, generated by a model, while actor and flow stay frozen and the guarantees stay locked. The sequels go further up the stack, and you can already guess how. But one melt at a time.

One more thing, before you object that your team already caches, templates, and hard-codes things back down after they have proven out: yes. Melting has an inverse. Proven emissions get refrozen into artifacts, and the mature systems in this series breathe in both directions. That operator gets its own chapter, two books from now, and a rocket to go with it. For now, the arrow points one way.

1.5 Where Guarantees Have Traditionally Come From

If classical software's behavior was frozen, where did its *trust* come from? It is worth taking inventory, because everything on this list was invented to guard authored behavior, and in later chapters we will ask, item by item, which of them survive contact with generated behavior.

Start at design time, where classical engineering does almost all of its guaranteeing. **Compilers and type systems** refuse, before a single instruction runs, entire categories of programs that could misbehave: a guarantee so absolute we stopped noticing it is one. **Tests** freeze expected behavior into executable claims: given this state, `f` must produce that state. **Schemas and contracts** pin down the shapes data may take at every boundary (a discipline whose absence once converted a Mars probe into a very expensive meteor; that story opens Chapter 3). **Code review** is a guarantee made of colleagues.

Then the runtime residue: thinner, humbler, and telling. **Assertions** die loudly rather than proceed wrongly. **Transactions** promise all-or-nothing, so a failure mid-flight cannot leave the world half-changed. **Watchdog timers**, the direct ancestors of Apollo's restart logic, sit outside the loop with one job: if the process stops making progress, kill it and recover, no questions asked. **Idempotency** makes retries safe; **checksums** make corruption visible; **monitoring** makes everything else visible to the people holding the pager.

Notice the proportions. Classical software front-loads its guarantees: the heavy machinery (types, tests, proofs, review) operates on the frozen artifacts *before* execution, and the runtime keeps only a light residue, because deterministic replay of verified artifacts does not need much guarding. The Apollo software was tested, simulated, and inspected into the ground for years; the restart logic was the thin last line, and in the event, the thin last line was enough, *because* everything behind it held still.

Now watch what this implies, because it is the engineering thesis of the entire series and we have arrived at it honestly: **the amount of guaranteeing you need at runtime is inversely proportional to how much of your behavior was frozen in advance.** Melt a layer, and the verification that used to happen at design time has nowhere to go but into the running system. Determinism does not disappear when the body becomes generated. It *relocates*: out of the behavior, into the guarantees. Chapter 12 will build that runtime machinery in earnest. Every chapter between here and there is preparation for it.

1.6 The First Crack in the Ice

So here is the classical machine, complete: a frozen body, executed by a frozen actor, choreographed by a frozen flow, bounded by locked guarantees. It landed on the Moon. It runs your bank. It is, by any fair accounting, the most successful engineering pattern of the last century.

And its limitation is the same fact as its virtue: *the body can only contain decisions somebody already made.* Every branch anticipated, every case enumerated, every "what next?" answered in advance by a person at a desk. For sixty years, when the world presented a situation the authors had not foreseen, software had exactly two moves: fail, or do the wrong thing confidently.

Chapter 2 makes the third move. We take the loop you are about to build, specifically one line of it, and replace the hand-written transition with an inference call:

```
state = f(state)        # a person wrote f, in advance
state = model(state)    # the step is generated, at runtime
```

One line. It looks like a refactor. It is a phase change: the first artifact in our stack becoming an emission, with everything that implies for errors, trust, and that inventory of guarantees you just took. But not yet. First you are going to build the frozen version, properly, with your own hands, because you cannot melt what you have not built, and because everything about it that seems boring today is a property you will spend the rest of this book fighting to keep.

1.7 Lab: A Loop You Can Trust

Meet **Meridian**, the company we will be keeping for four books: a mid-market subscription business, ten thousand customers, and a churn number that has lately given its executives a reason to take early lunches. Meridian will grow more elaborate as we go. Today it has ten accounts and one need: a churn report, produced by a loop so plain you could explain it to a review board on a napkin.

Everything in this lab is deliberately, pointedly deterministic. Same input, same output, every run, forever. Savor that. It is the last chapter where it comes free.

```
"""Meridian churn report -- a loop you can trust.

Book I, Chapter 1 lab. Deterministic. Boring. That's the point.
"""

from dataclasses import dataclass, replace
from enum import Enum

# ---------------------------------------------------------------
# STATE -- everything the loop knows, in one place, immutable.
# ---------------------------------------------------------------

class Phase(str, Enum):
    LOAD = "load"
    COMPUTE = "compute"
    SEGMENT = "segment"
    REPORT = "report"
    DONE = "done"

@dataclass(frozen=True)          # yes, frozen. The word will follow us around.
class State:
    phase: Phase = Phase.LOAD
```

```
    step: int = 0
    accounts: tuple = ()
    churn_rate: float | None = None
    segments: tuple = ()
    report: str | None = None

# ------------------------------------------------------------------
# THE BODY -- what happens during a step. Hand-written, every line.
# ------------------------------------------------------------------

RAW = (
    {"id": "A-01", "plan": "basic",      "cancelled": True},
    {"id": "A-02", "plan": "basic",      "cancelled": True},
    {"id": "A-03", "plan": "basic",      "cancelled": False},
    {"id": "A-04", "plan": "pro",        "cancelled": True},
    {"id": "A-05", "plan": "pro",        "cancelled": False},
    {"id": "A-06", "plan": "pro",        "cancelled": False},
    {"id": "A-07", "plan": "pro",        "cancelled": False},
    {"id": "A-08", "plan": "enterprise", "cancelled": False},
    {"id": "A-09", "plan": "enterprise", "cancelled": False},
    {"id": "A-10", "plan": "enterprise", "cancelled": False},
)

def load(s: State) -> State:
    return replace(s, phase=Phase.COMPUTE, accounts=RAW)

def compute(s: State) -> State:
    cancelled = sum(1 for a in s.accounts if a["cancelled"])
    return replace(s, phase=Phase.SEGMENT, churn_rate=cancelled /
   len(s.accounts))

def segment(s: State) -> State:
    plans = sorted({a["plan"] for a in s.accounts})
    segs = tuple(
        (p,
         sum(1 for a in s.accounts if a["plan"] == p and a["cancelled"]) /
         sum(1 for a in s.accounts if a["plan"] == p))
        for p in plans
    )
    return replace(s, phase=Phase.REPORT, segments=segs)

def report(s: State) -> State:
    lines = [f"MERIDIAN CHURN REPORT  |  overall: {s.churn_rate:.0%}"]
    lines += [f"  {plan:<12} {rate:>4.0%}" for plan, rate in s.segments]
    return replace(s, phase=Phase.DONE, report="\n".join(lines))

# ------------------------------------------------------------------
# THE FLOW -- how steps are arranged over time. Also hand-written.
# ------------------------------------------------------------------

FLOW = {
    Phase.LOAD: load,
    Phase.COMPUTE: compute,
```

```
    Phase.SEGMENT: segment,
    Phase.REPORT: report,
}

# ---------------------------------------------------------------
# THE GUARANTEES -- what must remain true, no matter what.
# ---------------------------------------------------------------

MAX_STEPS = 10   # the watchdog. Our rendezvous-radar insurance.

def run(state: State = State(), flow=FLOW):
    trail = []
    while True:
        if state.phase is Phase.DONE:                       # success condition
            trail.append(f"step {state.step}: TERMINATED -- success")
            break
        if state.step >= MAX_STEPS:                         # watchdog
            trail.append(f"step {state.step}: TERMINATED -- watchdog "
                         f"(step budget exhausted in phase
   '{state.phase.value}')")
            break
        body = flow[state.phase]                            # <- the flow,
   frozen
        nxt = body(state)                                   # <- the body,
   frozen
        assert isinstance(nxt.phase, Phase), "unknown phase"  # <- a guarantee
        trail.append(f"step {state.step}: {state.phase.value} ->
   {nxt.phase.value}")
        state = replace(nxt, step=state.step + 1)
    return state, trail
```

Before running it, walk the anatomy, because this file is the whole chapter in ninety lines. The **state** is one frozen dataclass: every fact the loop knows, immutable, so each step's input and output can be captured whole.

The **body** is four pure functions, each taking a state and returning the next; note that they contain *all* of the intelligence and *none* of the control.

The **flow** is a table, literally a dictionary, mapping where-you-are to what-runs-next; the entire choreography of this program fits in four lines you can read aloud.

And the **guarantees** are three: a success condition, a step budget standing watch outside everything, and one assertion at the boundary. There is also a fourth, so pervasive it is easy to miss: the *trail*, an audit log built as the loop runs, because a loop you cannot replay is a loop you are merely hoping about.

Run 1, nominal:

```
--- RUN 1: nominal ---
step 0: load -> compute
step 1: compute -> segment
step 2: segment -> report
step 3: report -> done
step 4: TERMINATED -- success

MERIDIAN CHURN REPORT  |  overall: 30%
  basic         67%
  enterprise     0%
  pro           25%
```

Thirty percent churn, concentrated brutally in the basic tier. Meridian's executives were right to skip dessert. But the report is not the point of the lab. The point is Run 2, in which we re-create the rendezvous radar.

One component misbehaves, realistically and undramatically. The `compute` step does its arithmetic correctly but, through the kind of one-line oversight that survives code review every day, forgets to advance the phase:

```
def stuck_compute(s: State) -> State:
    cancelled = sum(1 for a in s.accounts if a["cancelled"])
    return replace(s, churn_rate=cancelled / len(s.accounts))   # phase unchanged. Oops.

SABOTAGED = {**FLOW, Phase.COMPUTE: stuck_compute}
```

Left alone, this loop violates the first obligation from section 1.1 (nothing changes each time around) and would run until the heat death of the process. It is not left alone:

```
--- RUN 2: one component misbehaves ---
step 0: load -> compute
step 1: compute -> compute
step 2: compute -> compute
step 3: compute -> compute
step 4: compute -> compute
step 5: compute -> compute
step 6: compute -> compute
step 7: compute -> compute
step 8: compute -> compute
step 9: compute -> compute
step 10: TERMINATED -- watchdog (step budget exhausted in phase 'compute')
report produced: None
```

Look at that trail. `compute -> compute -> compute`: a component drowning in place, exactly the signature the AGC's executive saw in 1969. And then the locked layer doing its one job: not fixing the fault, not understanding it, just *refusing to let it*

run forever, and leaving behind an honest account of what happened and where. The body failed. The guarantee held. Your first loop just survived its own 1202, and the flight recorder can prove it.

Two closing instructions for the lab, one practical, one in pencil.

Practical: extend it. Add a `budget_ms` guarantee alongside `MAX_STEPS`. Add a second termination reason. Serialize the final `State` to JSON and confirm a fresh process can resume from it. You have just implemented replay, and you will want it forever.

And in pencil: circle these two lines.

```
        body = flow[state.phase]     # <— the flow, frozen
        nxt = body(state)            # <— the body, frozen
```

The second line is where Chapter 2 applies the torch. The first is where Book III does. Keep the file.

Where We've Landed

A loop is a rule applied to state until a condition holds: the one structure that turns finite text into open-ended work, carrying two obligations, progress each step and a reachable end. Every loop, from cron to Eagle, decomposes into the same four layers: **body** (what a step does), **actor** (who does it), **flow** (how steps are arranged), **guarantee** (what must remain true).

In classical software all of the first three are **artifacts**, frozen before execution, which is precisely why its runtime guarantees could stay thin, why its emissions stayed boring, and why a young man in Houston could say GO: the entire trust model of twentieth-century software rests on behavior that holds still.

And the series' single transformation is now on the table: `G(x)` turns an artifact into an emission, and this book performs it on the body, knowing that whatever determinism we melt out of the behavior must be rebuilt, at runtime, in the guarantees.

What's Next

In 1966, while Bohm and Jacopini were proving what loops could do, a computer scientist at MIT wrote a few hundred lines of pattern-matching code and discovered, to his lasting horror, that people would pour their hearts out to it. In the next chapter we put a model inside the loop, and find out why the hardest part is not making it work, but knowing what "working" now means.

Steve Bales said GO because he could finish the sentence "the machine will now…" with every clause of it written by someone he could name. The rest of this book is about earning that sentence back after we hand the pen to the machine.

CHAPTER 2

The Ghost in the Loop

Putting a model inside

> *The question is not whether machines think, but whether people can stop themselves from believing they do.*

In 1966, at MIT, a computer scientist named Joseph Weizenbaum built a program to make a point about how shallow machine conversation really was. It took him a few hundred lines of pattern matching. He called it ELIZA, after Eliza Doolittle, the flower girl who could be taught to speak like a duchess, and he gave it a script named DOCTOR that imitated a particular school of psychotherapist: the kind that mostly turns your own words back on you. Tell it *my boyfriend made me come here* and it might reply *your boyfriend made you come here?* Tell it you are depressed and it would ask you to say more about that. There was no understanding anywhere in the machine. Keyword, rank, decompose, reassemble, print. A parlor trick, built to expose parlor tricks.

Then his secretary sat down at the terminal. She had watched him build the thing. She knew, in the way you know about a colleague's project, exactly what it was. After a few exchanges she asked Weizenbaum to leave the room, because the conversation had become private.

He never fully recovered from that. People who knew precisely what ELIZA was confided in it anyway; some asked whether the sessions were being recorded, and objected. Weizenbaum spent much of the rest of his career, including an entire anguished book, warning the world about what he had accidentally demonstrated: that humans will extend trust, intimacy, and the presumption of a mind to anything that holds up its end of a conversation. The phenomenon still bears his program's name. Researchers call it the ELIZA effect.

Here is the detail that matters for us, the one that makes ELIZA the perfect front door to this chapter rather than just a good story. ELIZA was *frozen*. Completely, provably, line-by-line frozen. Every response it ever gave was the deterministic output of rules

a person had written; you could take any exchange, however uncanny, and trace it back to a numbered pattern in the DOCTOR script. In the vocabulary you earned in Chapter 1: the body was an artifact, the flow was an artifact, and the ghost was not in the machine at all. The ghost was in the reader.

Sixty years later the situation has inverted, and the inversion is this book's subject. The systems we are about to build really do generate their behavior at runtime. The transition function is no longer a page of rules anyone wrote; it is the output of a model, produced fresh each step, for states no author enumerated. The ghost has, in a meaningful engineering sense, moved into the machine. And so our problem is the mirror image of Weizenbaum's. His nightmare was people trusting a frozen script because it sounded alive. Ours is building the machinery that lets us justifiably trust something genuinely fluid, for reasons we can name, log, and defend.

In this chapter we perform the melt. We take the loop you built in Chapter 1, replace one line, and then spend the rest of the chapter honestly confronting what that one line costs: a new species of error, a new job for every guarantee, and a new definition of the word "working."

2.1 One Line of Difference

Put Chapter 1's loop core side by side with its melted successor:

```
# Chapter 1                              # Chapter 2
body = flow[state.phase]                 decision = model(render(state))
nxt = body(state)                        nxt = apply(decision, state)
```

On the left, a table you wrote decides what happens next, and a function you wrote does it. On the right, the state is rendered into a context, a model reads that context and *emits a decision*, and the shell applies it. The loop skeleton is identical: same while, same state, same termination checks standing guard. What changed is the answer to one question, the question this whole book orbits: *who decides what happens next?* Yesterday, you did, at your desk, in advance, for every case you could think of. Now the running system does, at runtime, for the case actually in front of it.

Notice what did *not* melt, because the restraint is the engineering. The state model: yours. The inventory of available actions: yours. The rendering of state into context: yours. The parsing and validation of what comes back: yours. The termination authority: emphatically yours. We have taken the classical loop and hollowed out exactly one chamber, the decision, and filled it with inference. Everything surrounding that chamber is as frozen as it was on Eagle.

Wait. Is choosing the next action not control flow?

A sharp reader will object that deciding what happens next sounds like flow, which this book promised to keep frozen. The distinction is worth stating precisely, because two entire sequels live on the far side of it. The *flow* here is the authored skeleton: one decision per iteration, a fixed vocabulary of actions, a fixed termination authority, a shell whose structure never changes shape at runtime. The model fills a slot *inside* each step. It cannot add a branch to the program, spawn a second loop, restructure the computation, or overrule the watchdog. When the skeleton itself becomes something the system generates, that is a different transformation with different dangers, and it gets its own book. For now: the model decides *within* the step. The arrangement *of* steps stays yours.

2.2 A Policy, Not an Oracle

You have almost certainly called a model before: paste in a document, get a summary, done. That is a model used as an *oracle*: a stateless function, consulted once, whose mistakes are annoying but contained. The moment you place the same model inside a loop, it becomes something categorically different: a *policy*, a standing rule for choosing actions, whose outputs become the inputs of its own future.

That feedback arrow changes everything, in both directions at once.

Downward first. Errors compound. An oracle's wrong answer sits inertly on the screen; a policy's wrong action changes the state, which changes the next context, which conditions the next decision. One bad step and the model is now navigating a world that its own mistake created, drifting further from anything its training ever resembled. Left unattended, a looped model does not make one error; it makes an error and then builds on it, with the fluency of something that has never once experienced doubt.

But the same arrow points upward. Feedback is also how the system gets to *notice*. An oracle can never discover it was wrong; there is no afterward. A policy acts, observes the consequence, and gets a fresh chance to respond to reality rather than to its expectation of reality. Every self-correcting behavior you have seen an agent perform, every retried command and revised approach, is this arrow doing its good work. The loop is what turns a text generator into a participant: something that can be wrong *and then find out*.

Two sober footnotes before we move on. First, whatever multiplies also multiplies costs: a policy consulted thirty times per task incurs thirty times the latency and

thirty times the bill of an oracle, a fact with an entire chapter of consequences waiting at the end of this book. Second, and write this somewhere visible: the *system* is not the model. The system is the model plus the shell plus the accumulated state, and it is the system, never the raw model, that your users experience and your auditors examine. Most of what the industry calls model failures are, on inspection, system design failures wearing the model as a mask.

2.3 Two Species of Wrong

Chapter 1's machine could fail, and did, in front of half a billion people. But recall *how* it failed: loudly, legibly, and by the book. The 1202 was a machine announcing its own distress with a numbered code that a person had assigned to that exact condition in advance. Classical failures are like that as a family. They are deterministic, so the same input reproduces them on demand. They are diagnosable, because a stack trace points at authored lines. And they are *fixable at the source*: find the wrong line, change it, and that failure is gone from the universe.

Model errors are a different species, and the difference is the intellectual adjustment this chapter is really asking of you. A model's error arrives fluent, well-formatted, and confident, indistinguishable in tone from its correct output. It may not reproduce: sample again and the error is gone, or different. It shifts with context in ways no line of code explains, because there is no line of code; there are weights, and you do not get to edit them. A program that is wrong crashes or contradicts itself. A model that is wrong *testifies*. Programs fail like machines. Models fail like witnesses: sincerely, plausibly, and sometimes about things that never happened.

The practical taxonomy has four entries, and you will meet all four, in the flesh, in this chapter's lab:

Malformed output. You asked for JSON; you received an enthusiastic paragraph. The cheapest error, because it is mechanically detectable at the boundary.

Out-of-space output. Well-formed, and requesting an action that does not exist. The model has invented a capability, which sounds exotic until you watch it happen on a Tuesday.

Plausible-but-wrong output. Well-formed, in-vocabulary, and a bad idea: the wrong action for this state, chosen with perfect syntax. The most expensive entry, because no parser can catch it; only a verifier that knows what *good* means can.

Premature success. The model declares the task complete while the work sits unfinished. A special case of the previous entry, so common and so corrosive that it earns its own line and, in the lab, its own guardrail.

Now the adjustment itself, stated as bluntly as it deserves. With classical errors, your posture is *debugging*: hunt the cause, edit the source, eliminate the failure. With model errors, that posture is unavailable. There is no line to fix. Your posture becomes *bounding*: assume every species above will occur at some frequency forever, and build the surrounding machinery so that when they occur, they are detected, contained, and recovered from, with the incident on the record. You do not fix a witness. You cross-examine one. Which is why the rest of this chapter is about the courtroom.

2.4 How to Make a Poet Fill In a Form

A language model wants to write prose. That is not a flaw; it is the substance of the thing. Ask one what to do next and it will, by default, answer the way a thoughtful colleague answers: in sentences, with hedges, context, and the occasional flourish. Charming in a colleague. Useless to a shell that must now decide which function to call.

The amateur response is to accept the prose and go fishing in it with regular expressions. Do not do this. You will spend your career maintaining a parser for the world's most creative format, and you will still lose, because the format changes whenever the weather does.

The professional response is to shrink the doorway. You cannot stop the poet writing poetry, but you can hand the poet a form on which poetry has no field:

```
{"action": "compute", "reason": "accounts loaded, need overall churn"}
```

One object. One `action`, drawn from an enumerated vocabulary you published in the prompt. One short `reason`. That is the entire interface between the fluid filling and the frozen shell, and every property of it is doing deliberate work. The enumeration converts an open-ended generation problem into a selection problem, which models are markedly better at and verifiers can check mechanically: the reply either names a real action or it does not, a decidable question, answered in microseconds, with no judgment involved. And the `reason` field is not decoration. It is *decision provenance*: a contemporaneous note in the model's own words for why this step happened, written into the trail beside the action it explains. Six chapters from now,

when a trajectory goes somewhere strange, those little clauses will be the difference between an investigation and a shrug.

Modern APIs will meet you more than halfway here, with function-calling interfaces and schema-constrained decoding that make malformed output rare rather than routine. Use them. But treat them as reducing the frequency of boundary violations, never as eliminating the boundary. The shell validates everything that crosses, however it was produced, because the shell's authority cannot depend on the good manners of the thing it exists to bound. And when validation fails? You do the one thing you could never do to a crashed program: you hand the form back and say, politely and mechanically, *this was not valid, try again*. A retry against a fresh sample is a genuinely new draw, not a rerun of a deterministic bug. It is the first entry in a long catalog of repair moves that exist only on this side of the melt.

2.5 The Shell Stays Frozen

Assemble the pieces and a shape emerges, one you will build in every chapter from here to the end of the book. It is a sandwich, and only the middle layer is new:

```
frozen    render(state)  -> context          you wrote this
fluid         model(context) -> text         generated, each step
frozen    parse -> validate -> apply -> log  you wrote this too
```

Everything above and below the model call is classical software, held to Chapter 1's standards without apology. The top slice decides what the model gets to see: which facts of the state, in what form, with what instructions and what vocabulary of actions. The bottom slice decides what the model gets to *do*: parse the reply, check it against schema and vocabulary and postconditions, apply it to the state through functions you wrote, and record the whole exchange in the trail. In between sits the one component you did not author and cannot inspect, held on both sides by components you did and can.

Read the sandwich as a statement about responsibility and it says something almost legal: *the model proposes; the shell disposes.* Nothing becomes real in your system because a model said it. It becomes real because the shell, applying rules written in advance by an accountable person, accepted it. That sentence is the entire trust architecture of this book in miniature, and it is why the shell must stay boring. Every clever behavior you are tempted to add to the shell is a behavior you can no longer rely on when the filling misbehaves.

One more Chapter 1 echo, because it is load-bearing: the trail. In a deterministic loop, logging was good hygiene. Here it is closer to a constitutional requirement. The model's decision process is invisible; the trail is the only place the system's history exists in inspectable form. Every prompt rendered, every reply received, every rejection and retry and reason: written down, replayable, and beyond the model's reach. In Chapter 9 this sandwich grows up and acquires a proper name, the harness, and a production-grade feature list. The shape will not change. The shape is the point.

2.6 G(body), Named

Time to say formally what we have done, and to update the picture we drew on Eagle.

```
G(body):  artifact(body)  ->  emission(body)
```

The step decision, which for the entire history of software has been an artifact (authored, reviewed, versioned, frozen), is now an emission: generated at runtime, per state, by a model. Redraw the stack:

```
 layer      | Meridian, Chapter 2              | authored when?
------------+----------------------------------+----------------
 GUARANTEE  | schema, vocabulary, watchdog,    |  design time
            | postconditions, repair budget    |   (LOCKED)
 FLOW       | the shell's skeleton             |  design time
 ACTOR      | one loop, one model, one role    |  design time
 BODY       | the decision, each step          |  RUNTIME (FLUID)
```

One row melted, three still frozen, and look at what happened to the guarantee row: it got *longer*. That is Chapter 1's inverse law arriving on schedule. The determinism we melted out of the body did not vanish; it moved into the locked layer, where it now lives as schemas, vocabularies, postconditions, and budgets, verified at runtime because it can no longer be assumed at design time.

There is a second migration in the diagram, quieter and easy to miss. In Chapter 1, the intelligence lived in the bodies (four functions that knew things) and the control was a dumb table. Now the intelligence lives in the decision, and the four functions have been demoted to something new: a vocabulary of capabilities, dumb, reliable, and waiting to be chosen. Hold that thought. In Chapter 7 those demoted functions acquire their proper name, tools, and turn out to be one of the most consequential design surfaces in the entire field.

Which leaves the question every melt must answer: *should* you? `G` is an operator, not an obligation, and applying it tastefully is a skill this book intends to build in you.

The rubric, in first draft: **melt where enumerating the cases is harder than verifying the outcome; keep frozen whatever is cheap to compute and expensive to get wrong.** Meridian's lab is a working demonstration of both halves. The decision melted, because scripting every path through an open-ended analysis is exactly the enumeration problem models dissolve. The arithmetic did not melt, and never will: churn is a division, division is free, and a model computing percentages is a liability you volunteered for. We asked the model to decide *that* computing churn is what happens next. We did not ask it to compute churn. Most of the bad agentic software in the world sits on the wrong side of that italicized line.

2.7 Lab: Melting the Meridian Loop

Same company, same ten accounts, same report. One difference, and by now you can name it in the series' own vocabulary: the FLOW table is gone. Nothing in this file knows the order of operations. The order will be an emission.

The model in the listing is a scripted stand-in, so the lab runs anywhere, deterministically, with no API key, and so we can choreograph its failures on cue. Everything on the shell side is exactly what you would ship.

```
"""Meridian, melted: Chapter 1's loop with a model deciding the next step.

Book I, Chapter 2 lab. The shell is frozen. The filling is not.
"""

import json
from dataclasses import dataclass, replace

# ---------------------------------------------------------------
# STATE and ACTIONS -- carried over from Chapter 1, minus the FLOW
# table. The arithmetic stays frozen. Only the *decision* melts.
# ---------------------------------------------------------------

RAW = (
    {"id": "A-01", "plan": "basic",      "cancelled": True},
    {"id": "A-02", "plan": "basic",      "cancelled": True},
    {"id": "A-03", "plan": "basic",      "cancelled": False},
    {"id": "A-04", "plan": "pro",        "cancelled": True},
    {"id": "A-05", "plan": "pro",        "cancelled": False},
    {"id": "A-06", "plan": "pro",        "cancelled": False},
    {"id": "A-07", "plan": "pro",        "cancelled": False},
    {"id": "A-08", "plan": "enterprise", "cancelled": False},
    {"id": "A-09", "plan": "enterprise", "cancelled": False},
    {"id": "A-10", "plan": "enterprise", "cancelled": False},
)
```

```
@dataclass(frozen=True)
class State:
    step: int = 0
    accounts: tuple = ()
    churn_rate: float | None = None
    segments: tuple = ()
    report: str | None = None

def load(s: State) -> State:
    return replace(s, accounts=RAW)

def compute(s: State) -> State:
    cancelled = sum(1 for a in s.accounts if a["cancelled"])
    return replace(s, churn_rate=cancelled / len(s.accounts))

def segment(s: State) -> State:
    plans = sorted({a["plan"] for a in s.accounts})
    segs = tuple(
        (p,
         sum(1 for a in s.accounts if a["plan"] == p and a["cancelled"]) /
         sum(1 for a in s.accounts if a["plan"] == p))
        for p in plans
    )
    return replace(s, segments=segs)

def write_report(s: State) -> State:
    lines = [f"MERIDIAN CHURN REPORT  |  overall: {s.churn_rate:.0%}"]
    lines += [f"  {plan:<12} {rate:>4.0%}" for plan, rate in s.segments]
    return replace(s, report="\n".join(lines))

ACTIONS = {"load": load, "compute": compute,
           "segment": segment, "write_report": write_report}

# ---------------------------------------------------------------
# RENDER -- the frozen half of the shell, on the way in.
# ---------------------------------------------------------------

def render(s: State) -> str:
    facts = [
        f"accounts loaded: {bool(s.accounts)}",
        f"churn computed: {s.churn_rate is not None}",
        f"segments computed: {bool(s.segments)}",
        f"report written: {s.report is not None}",
    ]
    return (
        "You are running Meridian's churn analysis, one step at a time.\n"
        "Current state:\n  " + "\n  ".join(facts) + "\n"
        f"Available actions: {', '.join(ACTIONS)}, or finish.\n"
        'Reply with JSON only: {"action": "<name>", "reason": "<short>"}'
    )

# ---------------------------------------------------------------
# MODEL -- the fluid filling. Scripted here; see the sidebar to go
```

```
# live. A real model receives render(state) and returns text.
# ------------------------------------------------------------------

class ScriptedModel:
    def __init__(self, replies):
        self.replies = list(replies)
    def __call__(self, prompt: str) -> str:
        return self.replies.pop(0)

# ------------------------------------------------------------------
# THE SHELL -- parse, validate, apply, log. Frozen, every line.
# ------------------------------------------------------------------

MAX_STEPS = 12     # progress budget: applied actions (the watchdog, as before)
MAX_REPAIRS = 4    # patience budget: tolerated bad replies before we abort

def run(model, state: State = State()):
    trail, repairs = [], 0
    while True:
        if state.step >= MAX_STEPS:
            trail.append(f"step {state.step}: TERMINATED -- watchdog")
            break
        if repairs >= MAX_REPAIRS:
            trail.append(f"step {state.step}: TERMINATED -- repair budget
   exhausted")
            break

        raw = model(render(state))

        try:                                    # guardrail 1: well-formed
            decision = json.loads(raw)
            action = decision["action"]
        except (json.JSONDecodeError, KeyError, TypeError):
            repairs += 1
            trail.append(f"step {state.step}: GUARDRAIL -- malformed reply "
                         f"rejected (repair {repairs}/{MAX_REPAIRS})")
            continue

        if action == "finish":                  # guardrail 3: postcondition
            if state.report is None:
                repairs += 1
                trail.append(f"step {state.step}: GUARDRAIL -- premature finish
   "
                             f"rejected, no report exists "
                             f"(repair {repairs}/{MAX_REPAIRS})")
                continue
            trail.append(f"step {state.step}: finish -- TERMINATED -- success")
            break

        if action not in ACTIONS:               # guardrail 2: real action only
            repairs += 1
            trail.append(f"step {state.step}: GUARDRAIL -- unknown action "
```

```
                        f"{action!r} rejected (repair
  {repairs}/{MAX_REPAIRS})")
            continue

        state = replace(ACTIONS[action](state), step=state.step + 1)
        trail.append(f"step {state.step - 1}: {action} "
                     f"({decision.get('reason', '')})")
    return state, trail
```

Walk the shell before running anything, because its architecture is section 2.5 made executable. Two budgets stand at the top of the loop, and they are guarding different things: `MAX_STEPS` bounds *progress*, exactly as in Chapter 1, while `MAX_REPAIRS` bounds *patience*, a resource the deterministic loop never needed because a deterministic loop never talks back. Then three guardrails, one per detectable species from section 2.3: a parse boundary for malformed output, a vocabulary check for invented actions, and a postcondition on `finish` for premature success. Note the etiquette of rejection: a guardrail never crashes and never punishes. It logs, spends a repair, and goes around again, which means the very next `render(state)` shows the model an unchanged world, the mechanical equivalent of handing back the form.

Run 1, a well-behaved model:

```
--- RUN 1: a well-behaved model ---
step 0: load (no data in state yet)
step 1: compute (accounts loaded, need overall churn)
step 2: segment (break churn down by plan)
step 3: write_report (all numbers ready)
step 4: finish -- TERMINATED -- success

MERIDIAN CHURN REPORT  |  overall: 30%
  basic         67%
  enterprise     0%
  pro           25%
```

The same report as Chapter 1, and the trail even reads the same, with one addition that should stop you for a second: the parentheses. Each step now carries its reason, in the decider's own words, at the moment of deciding. And remember what is absent from the file entirely: nobody told this program the order. Load before compute, compute before segment: that ordering was generated, step by step, from a rendered description of the state. You are looking at the first emission of the series.

But Run 1 is the demo. Run 2 is the chapter. Same shell, and a model having a worse day: one malformed reply, one hallucinated action, one premature declaration of victory, the full taxonomy on parade:

```
--- RUN 2: the same shell, a worse day ---
step 0: GUARDRAIL -- malformed reply rejected (repair 1/4)
step 0: load (fine, JSON it is)
step 1: GUARDRAIL -- unknown action 'email_the_ceo' rejected (repair 2/4)
step 1: compute (back to the plan)
step 2: segment (by plan tier)
step 3: GUARDRAIL -- premature finish rejected, no report exists (repair 3/4)
step 3: write_report (apparently we were not done)
step 4: finish -- TERMINATED -- success

MERIDIAN CHURN REPORT  |  overall: 30%
  basic         67%
  enterprise     0%
  pro           25%
```

Read the trail like a flight recorder, because that is what it is. A reply that was not JSON: rejected at the boundary, one repair spent. An action that does not exist, `email_the_ceo`, invented from thin air with a perfectly plausible reason attached: rejected by the vocabulary, second repair. A claim of success with no report in the state: rejected by the postcondition, third repair. And then, the finding: **the final report is byte-for-byte identical to Run 1's.** A misbehaving model, inside a sound shell, produced exactly the outcome a well-behaved one did, three incidents on the record, one repair still in the bank. In Chapter 1, the guarantee's whole job was to stop a broken body from running forever. The job has grown: the guarantees now steer a fallible decider back onto the road, mid-journey, without ever once understanding the road themselves. That is the trade this book asked you to consider, shown in eleven lines of terminal output. The behavior became fluid. The outcome did not.

Going live

Replace the scripted stand-in with a real model by swapping one object, using the official SDK. Nothing else in the file changes, which is the entire argument of section 2.5:

```
import anthropic
client = anthropic.Anthropic()   # reads ANTHROPIC_API_KEY from the environment

def live_model(prompt: str) -> str:
    msg = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=200,
        messages=[{"role": "user", "content": prompt}],
    )
    return msg.content[0].text

final, trail = run(live_model)
```

Run it a handful of times and study the trails. The orders will sometimes differ, the reasons will always differ, and every run should still end in the same verified report. If a guardrail fires, congratulations: you have caught a wild specimen of section 2.3, and your shell handled it while you watched. Current model names and API details live at docs.claude.com.

Two closing instructions, in the Chapter 1 tradition.

Practical: extend the shell, not the model. Feed rejection reasons back into the next prompt and measure whether repairs drop. Add a precondition guardrail (`compute` should be refusable when no accounts are loaded) and script a model that trips it. Then write the nastiest test in the book so far: a scripted model that answers `finish` with a plausible reason before any work is done, forever, and confirm which budget catches it and what the trail says afterward.

And in pencil, circle two regions. The `render` function, currently six honest lines: Chapter 8 lives entirely inside it, and it will not stay six lines. And the three guardrails, currently catching what a parser can see: Chapters 10 through 12 are the story of teaching that layer to catch what only a verifier can.

Where We've Landed

One line melted, and the whole trust model reorganized around it. A model inside a loop is a policy, not an oracle: its outputs feed its future inputs, which is simultaneously how errors compound and how self-correction becomes possible, and it means the unit of engineering is the system (model, shell, state, trail), never the model

alone. Model errors are a second species of wrong: fluent, probabilistic, and unfixable at the source, so the working posture shifts from debugging to bounding, from eliminating failures to detecting, containing, and recovering from them on the record. The interface that makes bounding tractable is a shrunken doorway (schema, enumerated vocabulary, decision provenance), and the structure that enforces it is the sandwich: a frozen shell in which the model proposes and the shell disposes. Formally, we performed `G(body)`: the step decision became an emission while actor, flow, and guarantees stayed frozen, and, exactly as Chapter 1 predicted, the guarantee layer grew to absorb the determinism the body gave up. And the melt comes with a rubric, not a mandate: generate the decision where enumeration is harder than verification; keep frozen whatever is cheap to compute and expensive to get wrong.

What's Next

The shell you just built has a dangerously innocent line in it: `render(state)` assumed the state was worth rendering. In 1999 two teams of excellent engineers flew a spacecraft into Mars because a number crossed a boundary without its units, a lesson about observation and contracts that cost three hundred million dollars and opens the next chapter. Before a model can decide well, it has to know what the world actually looks like. That turns out to be a discipline of its own.

Weizenbaum's secretary trusted a frozen script because it sounded alive. Bales trusted a fluid situation because everything around it held still. Between those two kinds of trust sits the whole craft of this book: the secretary could not say why she believed; Bales could. Build shells that put you, permanently, on Bales's side of that line.

When the machine decides, what keeps it honest?

For fifty years, software worked because somebody wrote down what should happen next. Now a model can decide the next step. The software still runs. The guarantees need to be rebuilt.

Agentic Loops is a field manual for that new machine: how it decides, what it may touch, what it remembers, when it stops, how it fails, what it costs, and how you prove it did the right thing.

Thirteen chapters. Thirteen runnable labs. One system, built from nothing to production.

The behavior changes.
The guarantees hold.